

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – COMPARATIVE ANALYSIS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4 – Precision (BL4)	Assessment:	Comparative Analysis
Weightage:	30%	Total Marks:	100
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	K.MAHESH BABU	Reg. No.:	192525122
Section / Batch:	CSA1214/2025	Date:	25/08/26

Instructions to Students

- Identify the relevant concepts of L1, L2, and L3 cache, SRAM, DRAM, NAND Flash, MRAM, RRAM, virtual memory, paging, latency, bandwidth, and throughput.
- Analyze the memory requirements of different computer systems by considering cache levels, memory technologies, access latency, bandwidth, throughput, capacity, and energy consumption.
- Evaluate the performance characteristics of different memory technologies and determine their suitability for cache memory, main memory, secondary storage, and future hierarchical memory systems.
- Compute relevant performance measures such as access time, latency, bandwidth, throughput, hit/miss rate, average memory access time (AMAT), speedup, and energy efficiency wherever applicable.
- Interpret the results by assessing memory bottlenecks, performance improvements, technology trade-offs, and the suitability of each memory technology or memory hierarchy for a given application.

QUESTIONS

1. Analyze the impact of cache hit rate and miss rate on processor performance. Assess how improving cache locality and reducing cache misses can enhance memory access efficiency and overall execution time.
2. Evaluate the role of memory latency, bandwidth, and throughput in determining system performance. Analyze how variations in these parameters affect applications with different memory-access requirements.
3. Assess the effectiveness of hierarchical memory organization using registers, cache, main memory, and secondary storage. Analyze how differences in access speed, capacity, and cost influence the design of an efficient memory hierarchy.
4. Examine the role of virtual memory and paging in supporting applications that require more memory than the available physical RAM. Analyze the effects of page faults, page replacement, and paging overhead on system latency and throughput.
5. Evaluate different memory technologies for high-performance applications based on latency, bandwidth, throughput, capacity, energy consumption, and endurance. Recommend a suitable memory hierarchy for a selected application and justify the choice based on its performance requirements.

Code of Conduct

/ K.MAHESH BABU(192525122)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **K.MAHESH BABU(192525122)**

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Marks Evaluation:

Q. No	Scope of Items (20)	Comparison Depth (25)	Use of Evidence (20)	Critical Thinking (20)	Clarity of Presentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 25	/ 20	/ 20	/ 15	/ 100

Course Coordinator

Faculty Incharge

Q1. Analyze the impact of cache hit rate and miss rate on processor performance. Assess how improving cache locality and reducing cache misses can enhance memory access efficiency and overall execution time.

Answer

Cache memory is a small and high-speed memory located close to the CPU. It stores frequently accessed instructions and data so that the processor can access them faster than accessing main memory.

The major performance factors of cache are **cache hit rate** and **cache miss rate**.

Cache Hit

A **cache hit** occurs when the required data is found in the cache.

For example:

CPU → Cache → Data found → Cache Hit

A cache hit allows the CPU to obtain data quickly and improves processor performance.

Cache Miss

A **cache miss** occurs when the required data is not present in the cache.

The processor must then obtain the data from a lower memory level such as L2, L3, or main memory.

CPU → L1 Miss → L2/L3 → Main Memory

This increases the memory access time.

Hit Rate and Miss Rate

The cache hit rate is:

The miss rate is:

For example, if there are 1000 memory accesses and 950 are cache hits:

Therefore:

Effect on AMAT

Average Memory Access Time can be represented as:

Suppose:

- Hit time = 2 ns
- Miss rate = 5%
- Miss penalty = 50 ns

Then:

$$AMAT = 2 + (0.05 \times 50)$$

$$AMAT = 2 + 2.5$$

$$AMAT = 4.5 \text{ ns}$$

If the miss rate is reduced to 1%:

$$AMAT = 2 + (0.01 \times 50)$$

$$AMAT = 2.5 \text{ ns}$$

Thus, reducing the miss rate significantly reduces average memory access time.

Cache Locality

Cache locality improves the probability of cache hits.

1. Temporal Locality

If data is accessed once, it is likely to be accessed again soon.

Example:

```
for(i = 0; i < 100; i++)
```

```
    sum = sum + a[i];
```

The variable sum is repeatedly accessed, showing temporal locality.

2. Spatial Locality

If one memory location is accessed, nearby memory locations are likely to be accessed.

Sequential array access is an example:

$a[0] \rightarrow a[1] \rightarrow a[2] \rightarrow a[3] \rightarrow a[4]$

This allows cache blocks to be used efficiently.

Impact on Processor Performance

Condition	Effect
High hit rate	Faster execution
Low miss rate	Lower memory delay
Good locality	More cache hits
Poor locality	More cache misses

High miss penalty	Greater performance loss
-------------------	--------------------------

Conclusion

A high cache hit rate improves processor performance because the CPU can obtain data quickly. A high miss rate increases memory access time and processor waiting time. Improving **temporal and spatial locality** and reducing cache misses decreases AMAT and overall execution time. Therefore, efficient cache utilization is essential for high-performance processors.

Q2. Evaluate the role of memory latency, bandwidth, and throughput in determining system performance. Analyze how variations in these parameters affect applications with different memory-access requirements.

Answer

Memory performance depends on several characteristics. The three important characteristics are **latency, bandwidth, and throughput**.

1. Memory Latency

Memory latency is the time required to obtain data after a memory request is made.

Lower latency means the processor receives data faster.

It is particularly important for applications that perform many small or random memory accesses.

Examples:

- Database systems
- Operating systems
- Interactive applications
- Random-access workloads

2. Memory Bandwidth

Memory bandwidth is the amount of data that can be transferred between memory and the processor in a given period.

It is commonly measured in **GB/s**.

For example, if 20 GB of data is transferred in 2 seconds:

High bandwidth is important for applications that transfer large amounts of data.

Examples:

- Video processing
- Image processing

- Scientific computing
- AI/ML workloads

3. Throughput

Throughput represents the amount of useful work completed by a system during a specific period.

It can be measured as:

- Transactions/second
- Requests/second
- Instructions/second
- Data processed/second

High throughput means the system can complete more work in a given time.

Comparison

Parameter	Meaning	Important for
Latency	Time to receive requested data	Random accesses
Bandwidth	Maximum data transfer rate	Large data transfers
Throughput	Useful work completed per unit time	Overall performance

Application-Based Analysis

Database

Databases often perform random memory accesses.

Therefore, low latency is highly important.

Video Processing

Video processing continuously transfers large amounts of data.

Therefore, high bandwidth is important.

AI/ML

AI applications process large datasets and perform massive calculations.

Therefore, they benefit from:

- High bandwidth
- High throughput
- Low enough latency

Critical Analysis

A system with high bandwidth is not necessarily better for every application.

For example, a database performing many small random accesses may benefit more from low latency than extremely high bandwidth.

On the other hand, video processing may benefit more from high bandwidth because large blocks of data must be transferred continuously.

Conclusion

Latency, bandwidth, and throughput represent different aspects of memory performance. Latency is important for fast individual accesses, bandwidth is important for large data transfers, and throughput determines how much useful work the system can complete. Therefore, memory design should be selected according to the specific access requirements of the application.

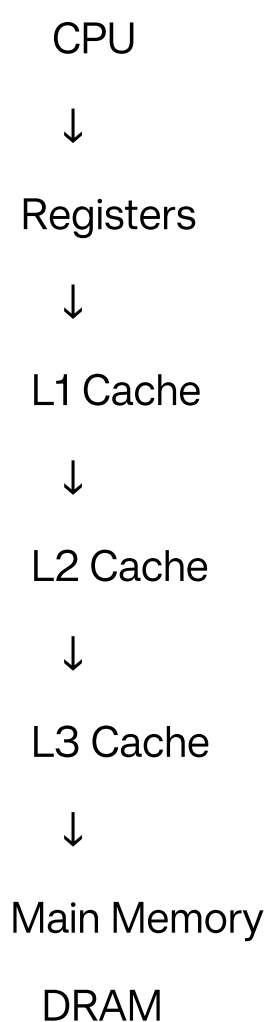
Q3. Assess the effectiveness of hierarchical memory organization using registers, cache, main memory, and secondary storage. Analyze how differences in access speed, capacity, and cost influence the design of an efficient memory hierarchy.

Answer

A computer uses a hierarchical memory organization because no single memory technology can provide extremely high speed, large capacity, low cost, and low energy consumption at the same time.

The memory hierarchy combines different types of memory according to their speed, capacity, and cost.

Memory Hierarchy





Secondary Storage

SSD / HDD

As we move from registers toward secondary storage:

- Access speed decreases
- Latency increases
- Capacity increases
- Cost per bit generally decreases

1. Registers

Registers are located inside the processor.

They provide extremely fast access and are used to store:

- Operands
- Instructions
- Addresses
- Intermediate results

However, their capacity is very small.

2. Cache Memory

Cache stores frequently used data and instructions close to the CPU.

It generally consists of:

- **L1 cache** – fastest and smallest
- **L2 cache** – larger and slower than L1
- **L3 cache** – larger and generally shared among processor cores

Cache reduces the number of accesses to slower main memory.

3. Main Memory

Main memory is usually implemented using **DRAM**.

It provides considerably greater capacity than cache at a lower cost per bit.

However, it is slower than registers and cache.

4. Secondary Storage

Secondary storage includes:

- SSD
- HDD

It provides very large storage capacity and is non-volatile.

However, its access time is much higher than RAM.

Comparison

Memory	Speed	Capacity	Cost/bit	Main Use
Registers	Highest	Very small	Very high	CPU operations
Cache	Very high	Small	High	Frequently used data
DRAM	Moderate	Large	Moderate	Main memory
SSD/HDD	Lower	Very large	Low	Permanent storage

Why Hierarchy is Effective

The hierarchy allows the system to keep frequently accessed data in faster memory and less frequently accessed data in larger, slower memory.

For example:

Frequently used data



Registers / Cache



Less frequently used data



DRAM



Long-term data



SSD / HDD

This provides a balance between:

- Performance

- Capacity
- Cost
- Energy consumption

Conclusion

Hierarchical memory organization is effective because it combines fast, small and expensive memory with slower, larger and less expensive memory. Registers and cache improve CPU performance, DRAM provides sufficient main-memory capacity, and secondary storage provides large persistent storage. Therefore, memory hierarchy is essential for achieving an efficient and economical computer system.

Q4. Examine the role of virtual memory and paging in supporting applications that require more memory than the available physical RAM. Analyze the effects of page faults, page replacement, and paging overhead on system latency and throughput.

Answer

Virtual memory is a memory-management technique that allows a system to provide an application with a larger logical memory space than the available physical RAM.

It uses secondary storage along with physical memory.

For example:

Application

↓

Virtual Memory

↓

Page Table

↓

Physical RAM

↓

Secondary Storage

Paging

Paging divides virtual memory into fixed-size blocks called **pages** and physical memory into blocks called **frames**.

Virtual Memory Physical Memory

Page 0 —————→ Frame 3

Page 1 —————→ Frame 0

Page 2 —————→ Frame 5

Page 3 —————→ Frame 1

The page table maintains the mapping between virtual pages and physical frames.

Page Fault

A **page fault** occurs when the processor requests a page that is not currently present in physical RAM.

The general process is:

CPU requests page



Page not in RAM



Page fault



OS locates page in storage



Page loaded into RAM



Page table updated



Execution continues

A page fault causes significant delay because accessing secondary storage is much slower than accessing RAM.

Page Replacement

If RAM is full and a required page needs to be loaded, the operating system must remove another page.

This is called **page replacement**.

Common algorithms include:

- FIFO
- LRU
- Optimal

Paging Overhead

Paging introduces additional processing because the operating system must perform:

- Address translation
- Page-table management
- Page-fault handling
- Page replacement
- Data transfer between RAM and storage

This increases memory access latency.

Effect on Throughput

If page faults are infrequent:

Few page faults → Less overhead → Higher throughput

If page faults are frequent:

Many page faults → More storage access → Lower throughput

Thrashing

When excessive page faults occur, the system may spend most of its time moving pages between RAM and storage instead of executing programs.

This condition is called **thrashing**.

Page Fault

↓

Load Page

↓

Another Page Fault

↓

Replace Page

↓

Another Page Fault

↓

Repeat

Thrashing greatly reduces system throughput.

Critical Analysis

Virtual memory provides flexibility because applications can use an address space larger than physical RAM.

However:

If an application's actively used data fits into RAM, performance can remain good.

If its active working set is larger than RAM, frequent page faults can severely reduce performance.

Conclusion

Virtual memory and paging allow large applications to execute even when their memory requirements exceed physical RAM. However, page faults and page replacement increase latency and reduce throughput. Excessive paging can lead to thrashing. Therefore, sufficient physical RAM and effective page-replacement strategies are important for maintaining system performance.

Q5. Evaluate different memory technologies for high-performance applications based on latency, bandwidth, throughput, capacity, energy consumption, and endurance. Recommend a suitable memory hierarchy for a selected application and justify the choice based on its performance requirements.

Answer

Different memory technologies provide different combinations of speed, capacity, cost, energy efficiency, and endurance. The major technologies considered are **SRAM, DRAM, NAND Flash, MRAM, and RRAM**.

Comparison of Memory Technologies

Technology	Latency	Capacity	Volatility	Endurance	Main Application
SRAM	Very low	Low	Volatile	Very high	Cache
DRAM	Low-moderate	High	Volatile	High	Main memory
NAND Flash	Higher	Very high	Non-volatile	Limited write/erase cycles	Storage
MRAM	Low	Moderate	Non-volatile	Very high	Emerging persistent memory
RRAM	Low-moderate	High potential	Non-volatile	Technology-dependent	Emerging memory

SRAM

SRAM provides very fast access and does not require periodic refresh.

Advantages:

- Very low latency
- High speed
- High endurance

Limitation:

- Expensive
- Low density
- Limited capacity

Therefore, SRAM is suitable for **CPU cache**.

DRAM

DRAM provides high capacity at a lower cost per bit than SRAM.

Advantages:

- Large capacity
- Suitable for main memory
- Good balance between cost and performance

Limitation:

- Requires refresh
- Slower than SRAM
- Volatile

Therefore:

NAND Flash

NAND Flash is non-volatile and provides high storage density.

Advantages:

- Large capacity
- Non-volatile
- Low cost per bit

Limitations:

- Higher latency than RAM

- Limited write/erase endurance

Therefore:

MRAM

MRAM is a non-volatile memory technology that uses magnetic states to store information.

Potential advantages include:

- Fast access
- Non-volatility
- High endurance
- Low standby power

It is suitable for emerging high-performance persistent-memory applications.

RRAM

RRAM stores information based on resistance states.

Potential advantages include:

- High density
- Non-volatility
- Low-power potential

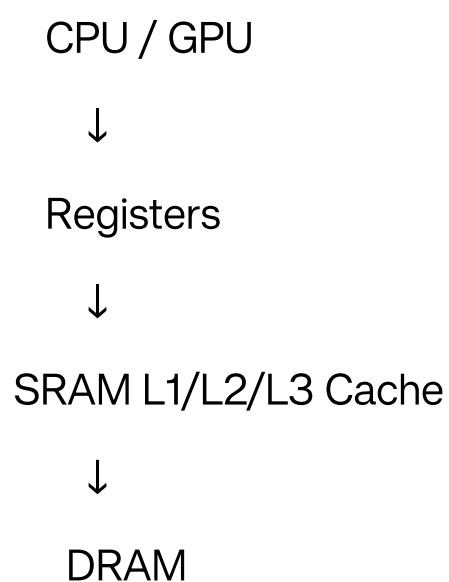
It is a promising technology for future memory systems.

Recommended Hierarchy

Selected Application: AI/ML Processing System

AI/ML applications process large datasets and require high bandwidth and throughput.

A suitable memory hierarchy is:





NAND Flash SSD



Large Data Storage

MRAM/RRAM can potentially be incorporated into future systems.

Justification

Registers

Used for immediate calculations.

Reason: Extremely low latency.

SRAM Cache

Stores frequently accessed instructions and data.

Reason: Reduces CPU/GPU waiting time.

DRAM

Stores active datasets and model parameters.

Reason: Provides much larger capacity than cache.

NAND Flash

Stores:

- Training datasets
- Models
- Checkpoints
- Other permanent data

Reason: High capacity and non-volatility.

Critical Evaluation

There is no single memory technology that provides the best:

- Latency
- Capacity
- Bandwidth
- Throughput
- Energy efficiency

- Endurance
- Cost

Therefore, a **hierarchical approach** is more effective.